

Week 02 Weekly Exercises

Objectives

- To practice using Rust's basic collection tyoes
- Exposure to simple derive macros
- Understand and use Rust's error handling philosophy, and the try trait

Activities To Be Completed

The following is a list of all the activities available to complete this week...

- **Tribonacci**
- **To Upper (Investigating strings!)**
- **Collections**
- **Data analysis**

The following practice activities are optional and are not **marked, or required** to be completed for the week.

None - all exercises this week are marked.

Preparation

Before attempting the weekly exercises you should re-read the relevant lecture slides and their accompanying examples.

Getting Started

Create a new directory for this week's exercises called `lab02`, change to this directory, and fetch the provided code for this week by running these commands:

```
$ mkdir lab02
$ cd lab02
$ 6991 fetch lab 02
```

Or, if you're not working on CSE, you can download the provided code as a [tar file](#).

EXERCISE:

Tribonacci

In this task, you will write a function that computes a variable number of [Tribonacci numbers](#), and also their sum.

Your program should read a single integer, `n`, from the command-line arguments, and subsequently compute the first `n` Tribonacci numbers, along with their sum.

If no command-line arguments are given, assume the value of `n` is assumed to be **10**.

Invalid input should be handled by returning a custom error message `"Please enter a valid size!"` (this is in the `error_msg` parameter of the `compute_tribonacci` function), through the struct `TribonacciError`, and then subsequently exiting the program.

HINT:

The [? operator](#) can be used to handle errors, and exit the program if an error occurs.

Try (🙄) using it to handle the error case of invalid input.

Click to reveal further hint

Your program need only be able to calculate from $n = 3$ to $n = 145$ inclusive. Note that $n = 145$ produces very large numbers that increase far past the width of a `i64` type. You may want to (strongly) consider using one of `i128` or `u128` instead, both of which can fit the largest number in that sequence.

You need only change the `compute_tribonacci` function.

```
$ 6991 cargo run -- 4
  Finished dev [unoptimized + debuginfo] target(s) in 0.00s
  Running `target/debug/tribonacci 4`
Values: [1, 1, 1, 3]

Sum: 6
$ 6991 cargo run -- 10
  Finished dev [unoptimized + debuginfo] target(s) in 0.00s
  Running `target/debug/tribonacci 10`
Values: [1, 1, 1, 3, 5, 9, 17, 31, 57, 105]

Sum: 230
$ 6991 cargo run -- -1
  Finished dev [unoptimized + debuginfo] target(s) in 0.00s
  Running `target/debug/tribonacci -1`
Error: Please enter a valid size
```

HINT:

Click to reveal further hint

You may find the below documentation, especially the examples, particularly helpful:

The documentation for the [args](#) function may be useful.

The documentation for the [vec](#) type may be useful.

The documentation for the [result](#) type may be useful.

The documentation for the [parse](#) method may be useful.

The documentation for the [map](#) method may be useful.

The documentation for the [map_err](#) method may be useful.

The documentation for the [sum](#) method may be useful.

When you think your program is working, you can use `autotest` to run some simple automated tests:

```
$ 6991 autotest
```

When you are finished working on this exercise, you must submit your work by running `give`:

```
$ 6991 give--crate
```

You must run `give` before **Week 3 Wednesday 21:00:00** to obtain the marks for this exercise. Note that this is an individual exercise; the work you submit with `give` must be entirely your own.

EXERCISE:

To Upper (Investigating strings!)

This activity is an investigation of Rust's strings!

You have been provided a simple crate, `to_upper`, which does not compile! Your task is to modify `to_upper/src/main.rs` so that it takes in a single command-line argument and converts the appropriate characters to uppercase. It may help to [RTFC](#)!

NOTE:

This exercise is based on [fasterthanlime](#)'s excellent article: [Working with Strings in Rust](#), which we certainly recommend reading.

Although quite long, reading it will leave you with a much deeper understanding of Rust's string types, their interactions with UTF-8, and how and why they actually work!

You will always be given valid input (e.g. no empty arguments).

```
$ 6991 cargo run -- 'hello world!!!'
    Finished dev [unoptimized + debuginfo] target(s) in 0.00s
    Running `target/debug/to_upper 'hello world'\!''\!''\!''`
arg = hello world!!!
upp = HELLO WORLD!!!
$ 6991 cargo run -- "uwu🥹👉👉"
    Finished dev [unoptimized + debuginfo] target(s) in 0.00s
    Running `target/debug/to_upper 'uwu🥹👉👉'`
arg = uwu🥹👉👉
upp = UWU🥹👉👉
```

When you think your program is working, you can use autotest to run some simple automated tests:

```
$ 6991 autotest
```

When you are finished working on this exercise, you must submit your work by running give:

```
$ 6991 give--crate
```

You must run give before **Week 3 Wednesday 21:00:00** to obtain the marks for this exercise. Note that this is an individual exercise; the work you submit with give must be entirely your own.

EXERCISE:

Collections

Rust's collection types could be generally categorised into four groups:

- Sequences
 - Vec
 - VecDeque
 - LinkedList
- Maps
 - HashMap
 - BTreeMap
- Sets
 - HashSet
 - BTreeSet
- Misc.
 - BinaryHeap

In this activity, we'll be exploring the performance of first three (Vec, VecDeque, and LinkedList), and HashMap collections.

For each of the four collections above, your code should:

- Create a new instance of the collection.
- Add some MAX_ITER elements to it.
- Remove MAX_ITER elements from it.
- Time each of the above operations.

NOTE:

For LinkedList insertion, you should insert at the end of the list. You should remove from the start of the list.

For HashMap you only need to insert at key i where i is the current iteration. Similarly, you should remove from key i.

Before you start, please write down some of your expectations about what you think will be the most performant collection for these operations.

The analysis of Vec and VecDeque has already been completed for you in the provided code.

This exercise also contains a theory section. At the end of your main function, you should write a comment that answers the following questions:

- Which collection type was the fastest for adding and removing elements?
 - Why do you think this was the case?
- Is there any significant difference between Vec and VecDeque deletion?
 - If so, why? If not, why not?
- When would you consider using VecDeque over Vec?
- When would you consider using LinkedList over Vec?
- Did the results surprise you? Why or why not?.

The above will be manually marked. There will be an opportunity to discuss this with a tutor and get marked during your week 4 workshop. If you cannot attend, your work will instead be manually marked (offline) during week 5.

HINT:

The docs are extremely helpful here! Reading (not just copying) the `std::collections` module docs will help greatly with **understanding** of this exercise.

The documentation for the [std::collections](#) module may be useful.

The documentation for the [std::collections::VecDeque::remove](#) method may be useful.

When you are finished working on this exercise, you must submit your work by running `give`:

```
$ 6991 give-crate
```

You must run `give` before **Week 3 Wednesday 21:00:00** to obtain the marks for this exercise. Note that this is an individual exercise; the work you submit with `give` must be entirely your own.

EXERCISE:
Data analysis

In this exercise, you are taking on the role of a prospective data analyst and have gotten your hands on a dataset of **(fictional!)** CSE enrolments.

The dataset is in the file `enrolments.psv` and each row contains the following columns (in order):

- UNSW Course Code
- UNSW Student Number
- Name
- UNSW Program
- UNSW Plan
- WAM
- UNSW Session
- Birthdate
- Sex

Each row of data represents one enrolment. One student may have multiple rows, as they may be enrolled in multiple courses.

You've been asked to process the data, and output answers to the following questions:

1. How many unique students are there?
2. What is the most common course (with how many students)?
3. What is the least common course (with how many students)?
4. What is the average WAM of all students in CSE (to 2 decimal places)?

Note that you can assume that there will be no two equally most-common or equally least-common courses in the dataset -- i.e. there will always be exactly one answer.

WARNING:

The contents of the `enrollments.psv` used for your program will change in marking

```
$ 6991 cargo run
  Finished dev [unoptimized + debuginfo] target(s) in 0.01s
   Running `target/debug/data_analysis`
Number of students: 8065
Most common course: COMP1511 with 953 students
Least common course: BIOM9002 with 1 students
Average WAM: 61.94
```

NOTE:

Authors Note

HINT:

This is a substantial exercise, break it down into steps! Try and avoid for-loops, and practice using iterator functions.

Click to reveal further hint

The documentation for the [csv](#) crate may be useful.

Click to reveal further hint

Reading CSV into Structs resources

The [seek method](#) may be useful to reset the reader to the start.

The documentation for the [serde](#) crate may be useful.

Click to reveal further hint

The documentation for the [HashMap](#) type may be useful.

The documentation for the [entry](#) method may be useful. The example is of particular note.

The documentation for the [max_by_key](#) method may be useful. Similarly, `min_by_key` is also useful.

The documentation for the [sum](#) method may be useful. This is useful for calculating the average.

When you think your program is working, you can use `autotest` to run some simple automated tests:

```
$ 6991 autotest
```

When you are finished working on this exercise, you must submit your work by running `give`:

```
$ 6991 give--crate
```

You must run `give` before **Week 3 Wednesday 21:00:00** to obtain the marks for this exercise. Note that this is an individual exercise; the work you submit with `give` must be entirely your own.

Submission

When you are finished each exercise make sure you submit your work by running `give`.

You can run `give` multiple times. Only your last submission will be marked.

Don't submit any exercises you haven't attempted.

If you are working at home, you may find it more convenient to upload your work via [give's web interface](#).

Remember you have until **Week 3 Wednesday 21:00:00** to submit your work.

You cannot obtain marks by e-mailing your code to tutors or lecturers.

Automarking will be run several days after the submission deadline, using test cases different to those `autotest` runs for you. (Hint: do your own testing as well as running `autotest`.)

After automarking is run you can [view your results here](#).

Weekly exercise Marks

When all exercises of a week are automarked you should be able to view the the marks [via give's web interface](#) or by running this command on a CSE machine:

```
$ 6991 classrun --sturec
```

COMP6991 23T1: Solving Modern Programming Problems with Rust is brought to you by
the [School of Computer Science and Engineering](#)
at the [University of New South Wales](#), Sydney.

For all enquiries, please email the class account at cs6991@cse.unsw.edu.au

CRICOS Provider 00098G

[Login as tutor](#)